




# SMSRLY - Property Recommendation Platform
## Final Digital Egypt Pioneers Project
## Technical Documentation

**Group:** ALX4_SWD5_S1
**Team Number:** T1
**Team Members:**
Abdelrhman Mohammed Elbrens
Huda Ali Fouad Mohamed
Mai Yasser Sobhy
Mahmoud El Sayed Ali
Raghad Ramzy
Loay Adel Ebrahim

# 1. Project Planning and Management

## Project Proposal:

### Executive Summary:

A common issue existing nationwide is the need for a temporary, short-term residence for but without having to deal with real estate agents or middlemen, often reputed as shady and exploitative. This proposal outlines a plan to establish a property recommender platform that seamlessly take user data and preferences as input and deliver back personalized property recommendations to the users, with contact info linking straight to the property owners themselves. The platform is poised to deliver unbiased, geographically-relevant recommendations through a modern and responsive interface.

### Overview:

Nationwide, citizens looking for an affordable residence solution to serve their education needs (university, student internship… etc), career needs, or simply as personal residence, turn to short-term temporary property leases. Most of the time, these property finders have to deal with real estate agents and brokers, which comes with their own sets of issues and headache for the customers.

One infamous issue is that brokers might request for hidden fees later on, forcibly squeezing additional profit from customers out of the fear that their property sale gets spoiled. Another issue for the potential leasers is that they are bound to the broker's motives, which might result in inadequate recommendations for the finder. Moreover, perceptions on brokers and middlemen greatly vary between economic sectors which further complicated consolidated documented experiences on the brokers themselves. An independent survey has been done on former and potential leasers that documented this divide, as well as more obscure acts of defraud brokers often deploy.

In general, as customers are gradually alienated from dealing with brokers and as their preference for dealing with the property owners directly increases, a lack for a platform that directly connects buyers with owners is more noticeably realized.

### Solution:

The proposed solution is a platform that can present to potential buyers personalized recommendations for the properties according to their provided preferences and data, including geolocation data and metadata, alongside personal contact info of the recommended properties itself, eliminating the need for complicating brokerage stage. The envisioned platform will provide a friendly interface that can utilize the users preferences around their ideal properties and provide back safe and reliable recommendations for the best properties available.

The platform also excels in being able to save their users cost, being a universally adoptable alternative to real estate brokers, and save time by connecting customers to property owners directly.

## Objectives:

The main goals needed to be achieved within 6 months of product launch are:
- 250,000 new accounts on the platform.
- Recommendation accuracy increased to 85%.
- Conversion rate of 10% (about 25 thousand confirmed property reservations on the platform).
- Reduction of the average expected time for renting a residential unit by 2 weeks.
- Stakeholder satisfaction rating of more than 80%.

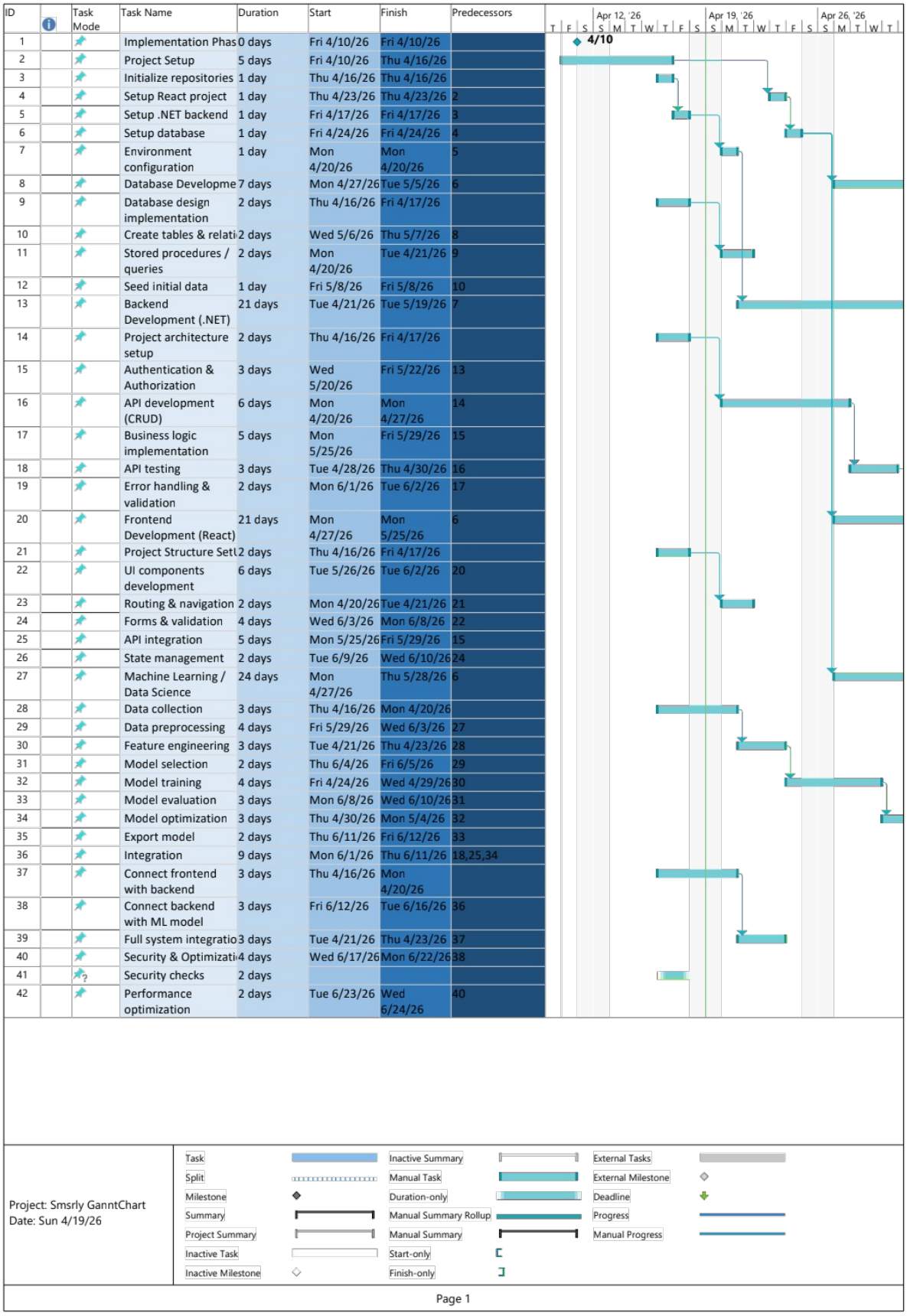
## Scope:

Elements of the platform included in-scope are:
- A complete, responsive web platform with attractive and functional UI/UX for listing properties with all their relevant details.
- Account registration, login, and management both as a tenant and owner of the project.
- Functional flows for accessing properties and contacting owners as a tenant, and managing properties as an owner.
- Functional back-end and database for property and user data storage and providing relevant endpoints for user interactions.
- Sufficient testing and database migrations for the back-end portion of the platform.
- Integration testing, quality assurance (QA), proper documentation, and deployment pipeline.

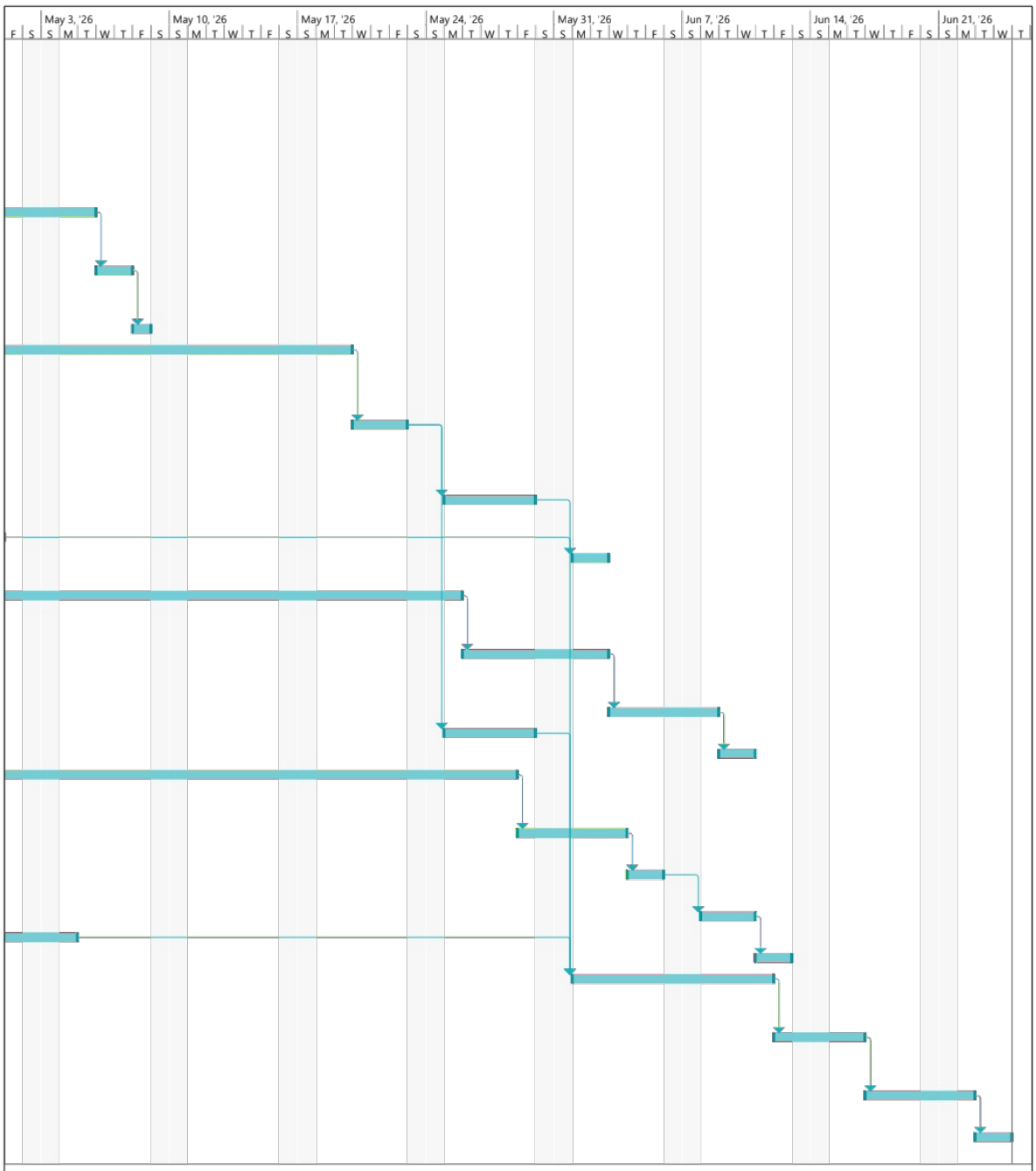
## Project Plan:

### Timeline:

This Gantt chart (2 pages) shows a detailed timeline of the entire project.



| ID | Task Mode | Task Name | Duration | Start | Finish | Predecessors |
|---|---|---|---|---|---|---|
| 1 | | Implementation Phas | 0 days | Fri 4/10/26 | Fri 4/10/26 | |
| 2 | | Project Setup | 5 days | Fri 4/10/26 | Thu 4/16/26 | |
| 3 | | Initialize repositories | 1 day | Thu 4/16/26 | Thu 4/16/26 | |
| 4 | | Setup React project | 1 day | Thu 4/23/26 | Thu 4/23/26 | 2 |
| 5 | | Setup .NET backend | 1 day | Fri 4/17/26 | Fri 4/17/26 | 3 |
| 6 | | Setup database | 1 day | Fri 4/24/26 | Fri 4/24/26 | 4 |
| 7 | | Environment configuration | 1 day | Mon 4/20/26 | Mon 4/20/26 | 5 |
| 8 | | Database Developme | 7 days | Mon 4/27/26 | Tue 5/5/26 | 6 |
| 9 | | Database design implementation | 2 days | Thu 4/16/26 | Fri 4/17/26 | |
| 10 | | Create tables & relati | 2 days | Wed 5/6/26 | Thu 5/7/26 | 8 |
| 11 | | Stored procedures / queries | 2 days | Mon 4/20/26 | Tue 4/21/26 | 9 |
| 12 | | Seed initial data | 1 day | Fri 5/8/26 | Fri 5/8/26 | 10 |
| 13 | | Backend Development (.NET) | 21 days | Tue 4/21/26 | Tue 5/19/26 | 7 |
| 14 | | Project architecture setup | 2 days | Thu 4/16/26 | Fri 4/17/26 | |
| 15 | | Authentication & Authorization | 3 days | Wed 5/20/26 | Fri 5/22/26 | 13 |
| 16 | | API development (CRUD) | 6 days | Mon 4/20/26 | Mon 4/27/26 | 14 |
| 17 | | Business logic implementation | 5 days | Mon 5/25/26 | Fri 5/29/26 | 15 |
| 18 | | API testing | 3 days | Tue 4/28/26 | Thu 4/30/26 | 16 |
| 19 | | Error handling & validation | 2 days | Mon 6/1/26 | Tue 6/2/26 | 17 |
| 20 | | Frontend Development (React) | 21 days | Mon 4/27/26 | Mon 5/25/26 | 6 |
| 21 | | Project Structure Setl | 2 days | Thu 4/16/26 | Fri 4/17/26 | |
| 22 | | UI components development | 6 days | Tue 5/26/26 | Tue 6/2/26 | 20 |
| 23 | | Routing & navigation | 2 days | Mon 4/20/26 | Tue 4/21/26 | 21 |
| 24 | | Forms & validation | 4 days | Wed 6/3/26 | Mon 6/8/26 | 22 |
| 25 | | API integration | 5 days | Mon 5/25/26 | Fri 5/29/26 | 15 |
| 26 | | State management | 2 days | Tue 6/9/26 | Wed 6/10/26 | 24 |
| 27 | | Machine Learning / Data Science | 24 days | Mon 4/27/26 | Thu 5/28/26 | 6 |
| 28 | | Data collection | 3 days | Thu 4/16/26 | Mon 4/20/26 | |
| 29 | | Data preprocessing | 4 days | Fri 5/29/26 | Wed 6/3/26 | 27 |
| 30 | | Feature engineering | 3 days | Tue 4/21/26 | Thu 4/23/26 | 28 |
| 31 | | Model selection | 2 days | Thu 6/4/26 | Fri 6/5/26 | 29 |
| 32 | | Model training | 4 days | Fri 4/24/26 | Wed 4/29/26 | 30 |
| 33 | | Model evaluation | 3 days | Mon 6/8/26 | Wed 6/10/26 | 31 |
| 34 | | Model optimization | 3 days | Thu 4/30/26 | Mon 5/4/26 | 32 |
| 35 | | Export model | 2 days | Thu 6/11/26 | Fri 6/12/26 | 33 |
| 36 | | Integration | 9 days | Mon 6/1/26 | Thu 6/11/26 | 18,25,34 |
| 37 | | Connect frontend with backend | 3 days | Thu 4/16/26 | Mon 4/20/26 | |
| 38 | | Connect backend with ML model | 3 days | Fri 6/12/26 | Tue 6/16/26 | 36 |
| 39 | | Full system integratio | 3 days | Tue 4/21/26 | Thu 4/23/26 | 37 |
| 40 | | Security & Optimizati | 4 days | Wed 6/17/26 | Mon 6/22/26 | 38 |
| 41 | | Security checks | 2 days | | | |
| 42 | | Performance optimization | 2 days | Tue 6/23/26 | Wed 6/24/26 | 40 |




Project: Smsrly GanntChart
Date: Sun 4/19/26

| | | | | | |
|---|---|---|---|---|---|
| Task | | Inactive Summary | | External Tasks | |
| Split | | Manual Task | | External Milestone | |
| Milestone | | Duration-only | | Deadline | |
| Summary | | Manual Summary Rollup | | Progress | |
| Project Summary | | Manual Summary | | Manual Progress | |
| Inactive Task | | Start-only | | | |
| Inactive Milestone | | Finish-only | | | |

**Milestones:**
Initial Project Planning: 4/17/2026
Front-end Component Development: 5/25/2026
Back-end Development: 5/29/2026
Machine Learning: 6/12/2026
Integration, Testing, and Optimization: 6/24/2026
Deployment and Launch: 6/31/2026

**Deliverables:**
The platform will be presented mainly as a web page, with functional components on both the front and back ends. Components of the platform to be delivered are as follows:

Project Plan and Visual Design:
Includes the initial planning and outline of the project, alongside wireframes and mockups for the project.
Deliverables: Project Plan, Mockups and Wireframes (Figma files and Images)

Frontend Development:
Responsive web pages according to the Figma planned files in the previous phases.
Deliverables: Unified git repository for both frontend and backend, including HTML, CSS, and JS code.

Backend Development:
Providing the backend for the web platform in synchrony with and integrating frontend development, complete with APIs, authentication, authorization components.
Deliverables: Unified git repository for both frontend and backend.

Machine Learning Model Building:
Machine learning model trained on proper property recommendations.
Deliverables: Custom-built libraries and recommender model code.

Integration Development and Testing:
Testing the web platform components as well as their integration with one another.
Deliverables: Integration testing report and execution log, integration code. QA report and results, alongside code patches and fixes as a whole.

Deployment and launch:
Production deployment and launch and handover of the entire site.
Deliverables: production deployment of the live site, documentation.

## Resource Allocation:

The following table shows the resource allocation of project members

| Resource Description | Role/Responsibilities | Number of People | Utilization Timeframe | Allocated Tasks |
|---|---|---|---|---|
| Team Leader | Oversees entire project | 1 | 4 Months | Planning, scheduling, and monitoring tasks |
| Database Administrators | Database creation and maintenance | 2 | 2 Weeks | Database design, creation, administration, and maintenance |
| Front-end Developers | Website front-end creation and maintenance | 3 | 1 Month | UI/UX design, front-end development |
| Back-end Developers | Back-end system creation and integration | 3 | 1 Month | Back-end creation, management, integration with front-end |
| Machine Learning Experts | Developing machine learning models | 2 | 1 Month | Develop recommender system code and relevant libraries |
| Testing and QA Engineers | Testing individual components and integration | 3 | 2 Weeks | Testing integrated website and reporting suggested maintenance and fixes |
| DevOps Engineer | Deployment and containerization | 1 | 1 Week | Deployment and proper containerization of website and recommender system stack |

## Risk Assessment and Mitigations:

| Risk | Probability | Impact | Mitigation |
|---|---|---|---|
| Unclear or changing requirements | High | High | Define requirements early-on, validate with supervisor weekly, freeze scope after approval |
| Time Management Issues | High | High | Create detailed schedule track progress weekly, keep buffer before deadlines |
| Integration issues between frontend, machine learning, backend | Medium | High | Develop and test each module separately then integrate step by step using APIs |
| Low accuracy of machine learning Models | Medium | High | Use quality datasets, try multiple models, perform tuning and evaluation |
| Poor data quality (missing or inconsistent data) | High | Medium | Apply data cleaning procedures, preprocessing, and validation techniques. |
| Backend or API faliure | Medium | High | Test APIs, implement error handling and validation |
| Front and back communication error | Medium | Medium | Use clear API contracts and consistent data format (JSON) |
| DB design error | Medium | High | Review ERD carefully, apply normalization and test queries early-on |
| Security Vulnerabilities (user data, authentication) | Medium | High | Implement JWT, authentication, validate all inputs, secure APIs |
| Performance issues under load | Medium | Medium | Optimize queries, reduce APIs calls and improve system efficiency |

| Risk | Probability | Impact | Mitigation |
|---|---|---|---|
| Deployment issues | Medium | Medium | Test deployment early, use stable hosting environment |
| Version Control Conflicts | Medium | Medium | Use a clear branching startegy, pull update before pushing changes |
| UI/UX issues affecting user experience | Medium | Medium | Design wireframes first, gather early feedback |
| Team coordination issues | High | Medium | Assign clear roles and hold regular meetings |
| Incomplete documentation | High | Medium | Write documentation continuously during development |

**Risk Mitigation Strategy:**

Project will follow these strategies to reduce risks:

- Divide the project into small, manageable tasks.
- Perform weekly progress tracking and reviews.
- Test each module immediately after development.
- Start integration early instead of delaying it.
- Maintain proper documentation through the project.

**Key Criteria Risk:**

Most critical risks are:

- Time management issues.
- System integration challenges.
- Machine learning model performance.

## Key Performance Indicatiors (KPIs)

### 1. System Performance KPIs

| KPI | Target | Measurement Method |
|---|---|---|
| API response time | Less than 2 sec | Backend logging, performance testing tools |
| System uptime | 99% availability | Hosting monitoring tools |
| Page load time | Less than 3 sec | Browser performance tools |
| Database query time | Less than 500 ms | Database profiling tools |

### 2. Functional KPIs

| KPI | Target | Measurement Method |
|---|---|---|
| Successful user registration rate | 100% functional flow | Testing scenarios |
| Successful property listing, creation | 100% successful rate | Functional testing |
| Successful property search result | Accurate results in top 90% cases | Manual and automated testing |
| Chat/Contact system success rate | 95% successful message delivery | System logs |

### 3. Machine Learning KPIs

| KPI | Target | Measurement Method |
|---|---|---|
| Model accuracy | Above 85% | Evaluation matrix (accuracy, F-1 score) |
| Prediction error | Less than 15% | Model evaluation reports |
| Training time | Optimizing within acceptable range | Training logs |
| Data processing rates | Less than 5 sec per request | Pipeline testing |

### 4. User experience KPIs

| KPI | Target | Measurement Method |
|---|---|---|
| User task completion rate | Above 90% | User testing session |
| User satisfaction score | Above 4/5 | Surveys or feedback forms |
| Navigation success rate | Above 90% | Usability Testing |
| Drop-off rate | Less than 20% | Analytics tracking |

## 5. Security KPIs

| KPI | Target | Measurement Method |
|---|---|---|
| Non-authorized access attempts blocked | 100% | System audit |
| Data encryption coverage | 100% sensitive data | |
| Authentication success rate | 100% valid users | Login system logs |

## 6. Project Delivery KPIs

| KPI | Target | Measurement Method |
|---|---|---|
| On-time task completion | 90% of tasks on schedule | MS Project tracking |
| Code coverage | Above 70% | Testing reports |
| Bug resolution time | Less than 48 hours | Issue tracking system |
| Documentation completion | 100% | Github repository review |

# 2. Literature Review

## Existing Systems and Platforms

### 1. Property Finder

Property Finder is one of the leading real estate platforms in the Middle East and North Africa. It provides users with the ability to search for properties using filters such as price, location, and property type. The platform mainly targets users who are looking to buy or rent properties through agents.

**Key features:**

- Advanced filtering system
- High-quality property listings with images and descriptions
- Location-based search

**Limitations:**

- It relies primarily on intermediaries in the communication process.
- It does not provide full direct interaction between the owner and the buyer.
- Some listings may be outdated or duplicated.

### 2. Dubizzle Real Estate

Dubizzle is a general-purpose online marketplace that includes a real estate section. Users can post advertisements for properties and communicate directly with interested buyers or tenants. Unlike specialized platforms, Dubizzle focuses on simplicity and wide accessibility.

**Key features:**

- Direct communication between users
- Easy and fast property listing
- Large and diverse user base

**Limitations:**

- There is no dedicated specialized system for managing real estate data.
- Weak filters and advanced search.
- No smart recommendation system
- The user experience is less organized compared to specialized platforms.

3. Aqarmap

Aqarmap is a specialized Egyptian real estate platform that provides detailed listings and market insights. It includes tools such as price estimation and map-based property search. The platform targets both buyers and investors.

**Key features:**

- Detailed property listings
- Market analysis and price guides
- Map-based search functionality

**Limitations:**

- Partially relies on intermediaries.
- It does not provide a full, direct communication system within the platform.
- Limited in the use of machine learning and predictive technologies
- Some operations take place outside the system.

## Research Studies

Recent studies have explored the use of artificial intelligence in real estate systems. For example, research on recommendation systems has shown that machine learning can improve property search by analyzing user preferences and behavior. These systems help users find relevant properties faster and reduce decision time.

Other studies focused on data analysis and prediction models for estimating property prices. These models rely on features such as location, size, and market trends. However, most research focuses on prediction and recommendation only. There is limited work on combining extensive machine learning with full real estate platforms that support direct user interaction without intermediaries.

## Comparative Analysis

From analyzing the above platforms and studies:

- Specialized platforms like Property Finder and Aqarmap provide structured data and better search
- General platforms like Dubizzle provide direct communication but lack organization
- Most systems either focus on usability or direct communication, but not both
- Machine learning integration is still limited or not fully utilized in user interaction

## Gap Analysis

- Heavy reliance on intermediaries across most platforms
- Lack of fully direct communication within the system
- Absence of a unified system that combines organization, ease of use, and direct communication
- Weak use of machine learning to enhance the search and recommended experience
- Some platforms suffer from inaccurate or outdated data

# 3. Requirements Gathering

## Stakeholder Analysis

### Stakeholder Identities:

The stakeholders involved in the project are divided into:
Internal stakeholders: The team members and project supervisors.
External stakeholders: Tenants dealing with the real estate brokers and agents, property owners, regulatory bodies facilitating real estate ownership and brokerage.

### Stakeholder Engagement and Feedback Collection:

A survey was made to gauge and measure the feedback of the external stakeholder, especially local tenants who tried and succeeded in renting a property unit (or failed to do so due to issues with brokers). The survey form was Arabic-based, and was mostly distributed to key target customers such as students and fresh graduate workers.
The responses was anonymous, with the only identifying pieces of information being gender, age category, and current occupation.

### Stakeholder Analysis:

The following trends were observed while analyzing the survey responses:
- About half of the respondents fall into the target category (at least tried to find a household before).
- While most of the target customers are of a specific target demographic (18-23 years old), some target customers were of an older age category with stable professions, a sign of interest from the older demographics.
- For the portion of target customers who already dealt with brokers/middlemen, more than 75% of them showed preference for directly dealing with the owner. Also, about a third of the target customers cited concrete issues with the brokers.
- For the target customers, Net Promoter Score (NPS) was about 3.86/5 stars. NPS for people who already rented property units was slightly higher, about 4.05/5 stars, in the range of cite-able interest for the project.

## User Stories and Use Cases

### User Stories:

Distinct user stories relevant stakeholders are most likely to say include:

- “As a tenant, I want to be able to search for rental units and select the one most relevant to my needs without resorting on other real estate brokers.”
- “As a property owner, I want to be able to list and manage my properties and put them up for rent without contacting and paying for any real estate brokers to handle the work for me.”
- “As a student, I want to be able to find the most affordable property for my university and contact their owners directly.”
- “As a platform admin, I want to manage and moderate interactions between tenants and owners, and manage new users on the platform.”

### Use Cases:

The two main use cases critical to the project’s success are as follows:

Use case 1: Tenant renting a residential unit

- Primary actor: Platform user (Tenant)
- Goals: Look for a suitable residential unit and contact its owner.
- Stakeholders: Tenants looking for a residential unit, primarily student and fresh graduate workers.
- Preconditions: User must be registered as a tenant.
- Post-conditions: User successfully finds the appropriate unit and directly contacts its owner on the platform.
- Flow:
    - User logs into the platform (or signs up if he is a new user).
    - System presents the user with a preliminary survey to bootstrap their personalized recommendations.
    - User fills the form with his preferences.
    - System presents the user with the personalized, relevant property suggestions.
    - User picks his appropriate residential unit, and the system presents him with the overview of the unit along with all of its details.
    - User is then able to call the owners or contact them directly on the platform.

Use case 2: Owner registering a property

- Primary actor: Platform user (Owner)
- Goals: Register their property on the platform
- Stakeholders: Property owners looking to rent their unit(s) to interested tenants.
- Preconditions: User must be registered as an owner.
- Post-conditions: User successfully registers their property on the platform
- Flow:
    - User logs into the platform (or signs up if he is a new user).
    - System presents the user with a dashboard to register and manage properties.
    - User chooses to add a new property.
    - System represents them with a form to fill in all the sufficient details for his added property.
    - After filling the survey, system informs the user of the successful registration, and user can now view his property listed and recommendable to users.

## Functional Requirements

- Users should be able to log in and register as either a tenant or an owner.
- The system should be able to list properties on demand and respecting personalization based on the user's preferences.
- Users should be able to search manually for a property and obtain accurate, idempotent results.
- Property owners should be able to manage properties with adding, deleting, or modifying each property they are confirmed to own.
- Tenants and owners should be able to message or contact each other directly on the platform.
- Admins should be able to manage and moderate users on the platform.

## Non-functional Requirements

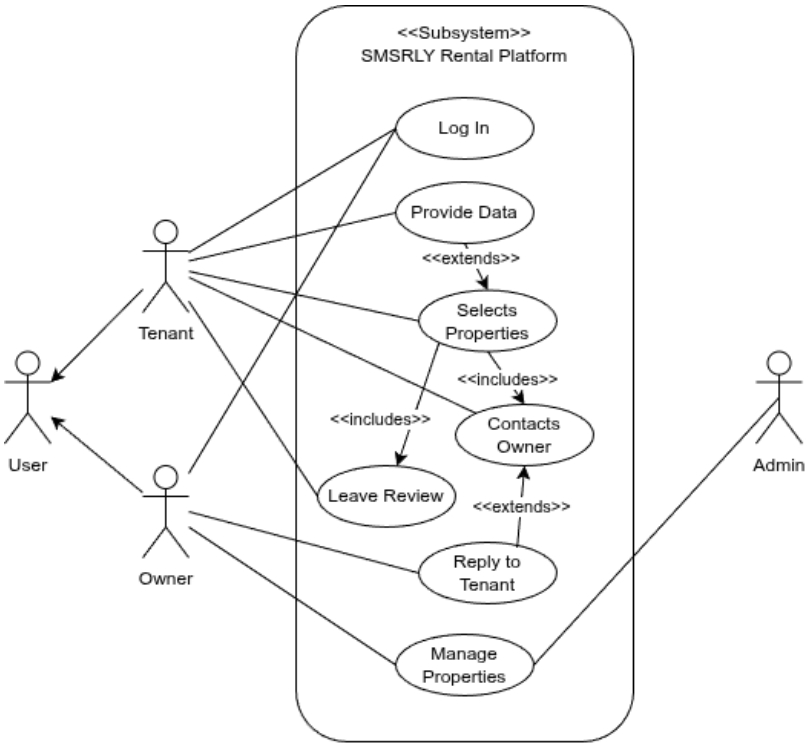
- The system should be fast and responsive, with minimal response and load time delays.
- The system should operate with high availability and reliability with minimal downtime.
- Property recommendations from the recommender systems should be accurate, reliable, and satisfactory to users.
- The entire system should have utmost security, with hardened data encryption, authentication, and authorization implemented whenever possible.
- The system should be highly scalable, accommodating for wider ranges of potential customers when available.
- The platform's interface should have a friendly and straightforward user experience while allowing wide range of controls over the property listings.

# 4. System Analysis and Design

## Problem Statements and Objectives

### Use Case Diagram

As documented in the previous phase in the use case and user stories sections, the main actors are the users (divided in implementation as tenants and owners,) and admins. The interactions between the users, admins, and the system are illustrated as follows:




### Functional Requirements

- Users should be able to log in and register as either a tenant or an owner.
- The system should be able to list properties on demand and respecting personalization based on the user's preferences.
- Users should be able to search manually for a property and obtain accurate, idempotent results.
- Property owners should be able to manage properties with adding, deleting, or modifying each property they are confirmed to own.
- Tenants and owners should be able to message or contact each other directly on the platform.
- Admins should be able to manage and moderate users on the platform.

## Non-functional Requirements

- The system should be fast and responsive, with minimal response and load time delays.
- The system should operate with high availability and reliability with minimal downtime.
- Property recommendations from the recommender systems should be accurate, reliable, and satisfactory to users.
- The entire system should have utmost security, with hardened data encryption, authentication, and authorization implemented whenever possible.
- The system should be highly scalable, accommodating for wider ranges of potential customers when available.
- The platform's interface should have a friendly and straightforward user experience while allowing wide range of controls over the property listings.

## Software Architecture

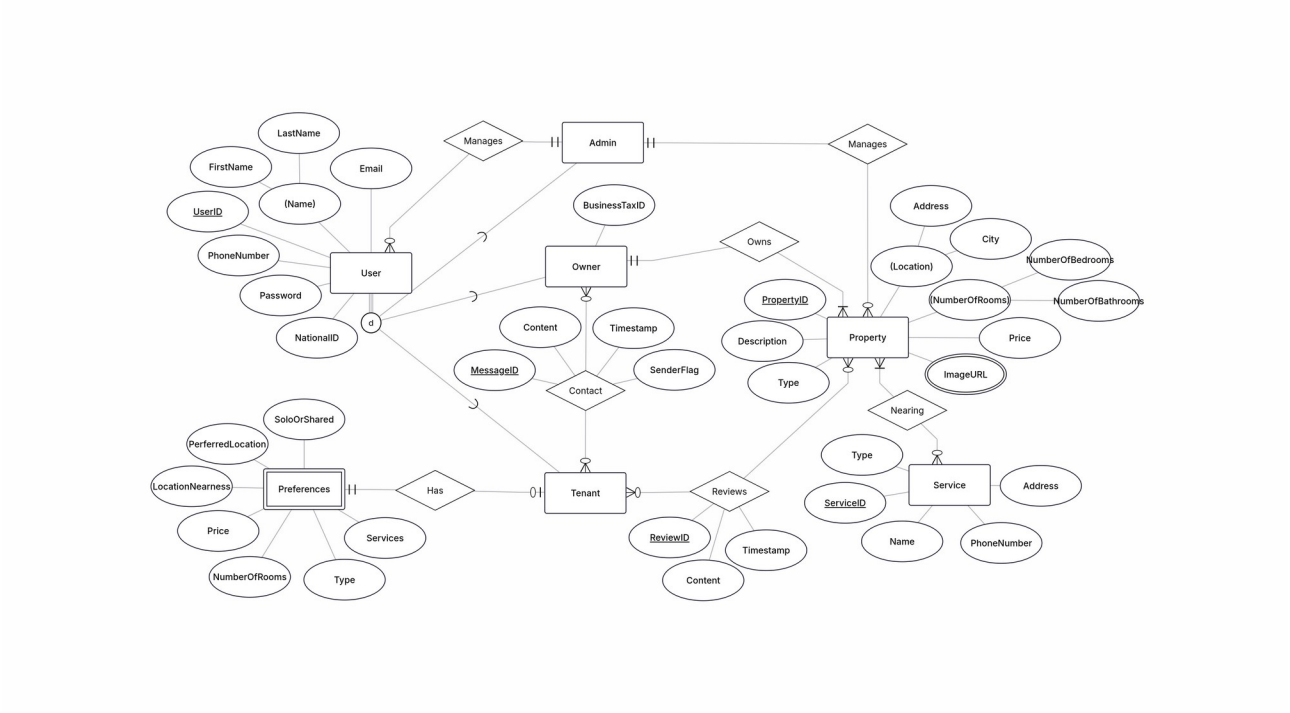
The back-end architecture of the web implementation of the system follows a 3-tier architecture plan, namely:

- Database layer: code pertaining to database definition and management, including database models and contexts.
- Business/Application layer: middle tier pertaining to processing data and handing it to the following layer, named presentation layer.
- Interface layer: code pertaining to exposing and presenting processed data via interfaces and endpoints (APIs).

This architecture can also be retroactively applied to the entire platform stack, where the application layer represents both business and interface layers in the back-end architecture, while a separate presentation layer pertains to the front-end development of the platform.

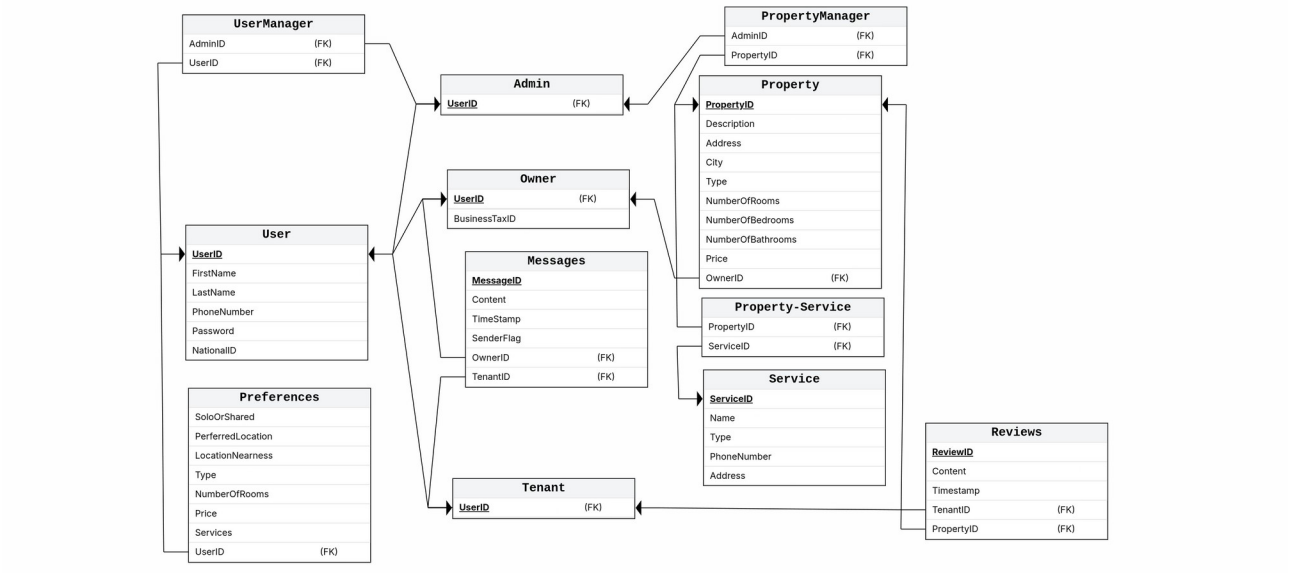
# Database Design and Data Modeling

## Entity-Relation Diagram (ERD)




The diagram contains advanced features in its implementation, such as the differentiation between the supertype (User) and its subtypes (Tenant, Owner, and Admin).

## Logical and Physical Schema

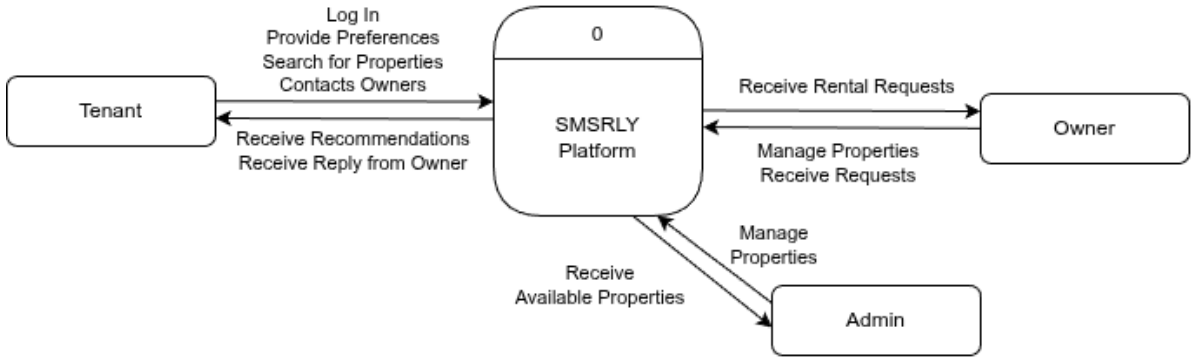



The schema is planned to be implemented in a code-first fashion, codifying the schema directly into the data access tier of the software architecture.

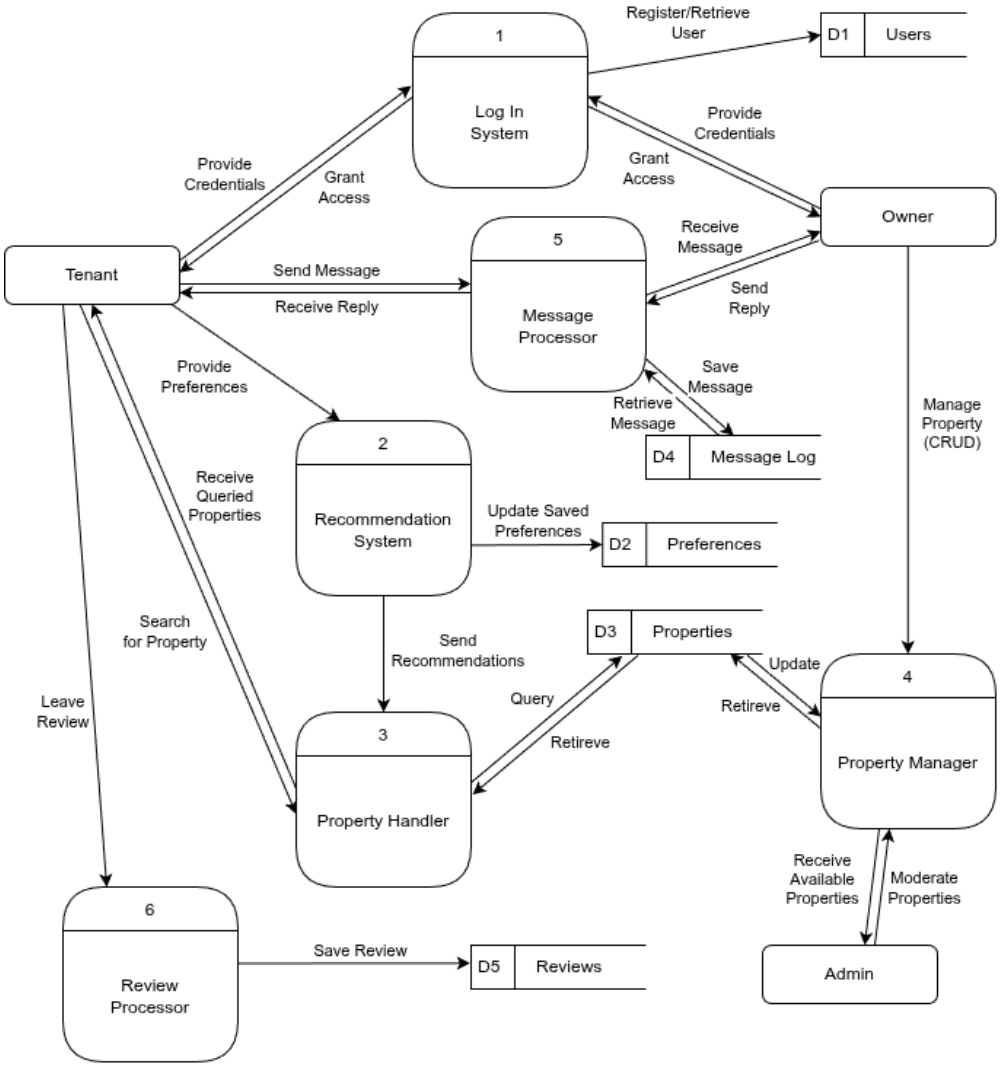
## Data Flow and System Behavior

### DFDs (Data Flow Diagrams)

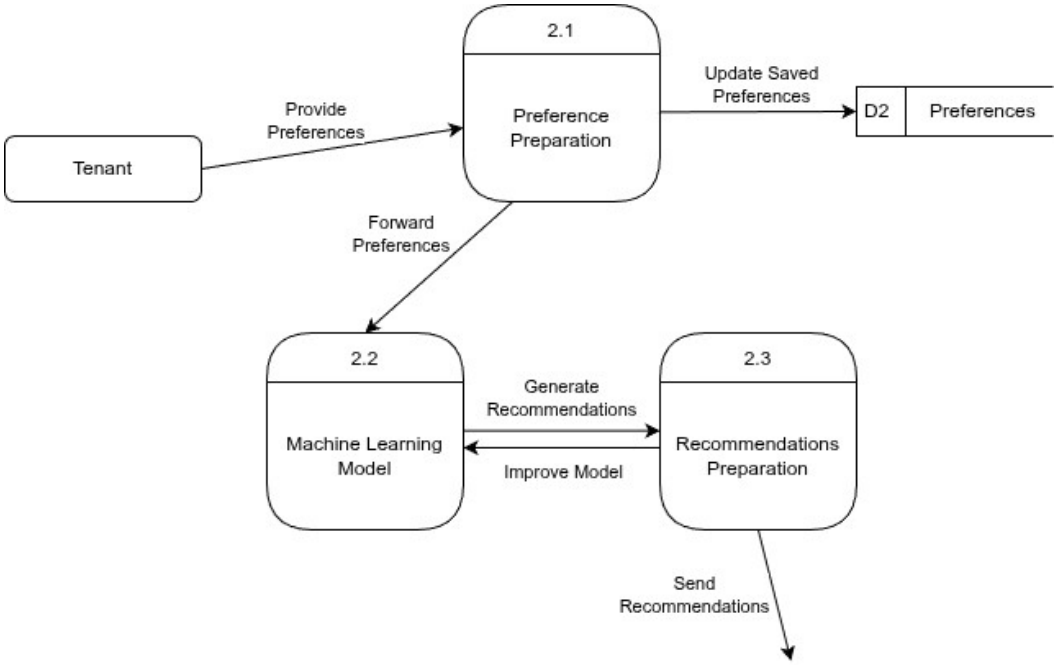
Context Diagram:



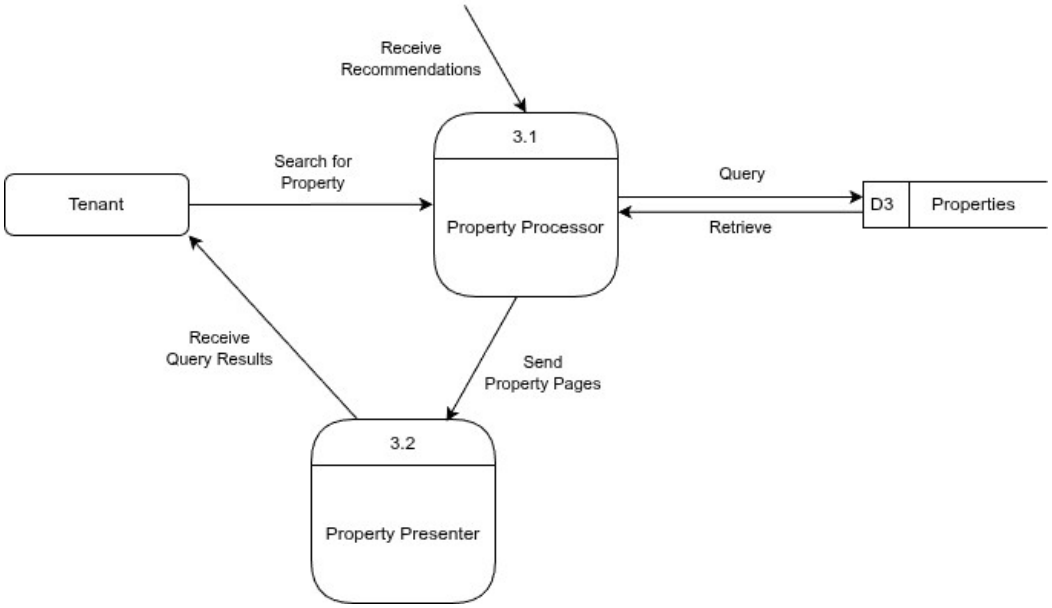

Level-0 Diagram:



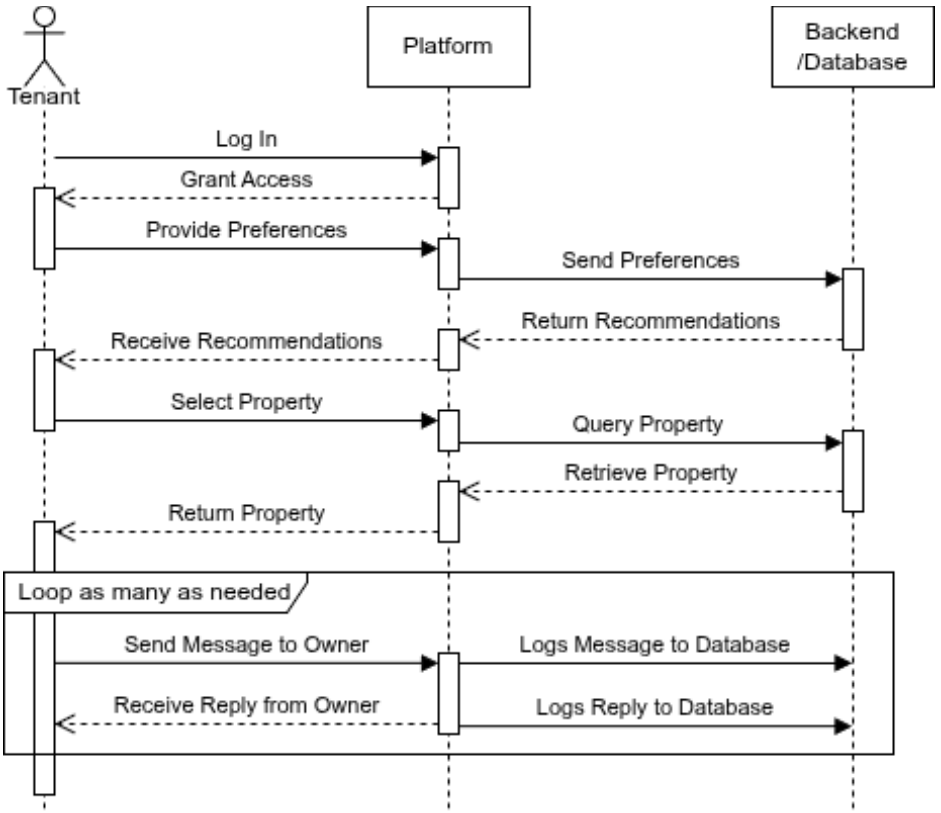
Child Diagram for Process 2 (Recommendation System):



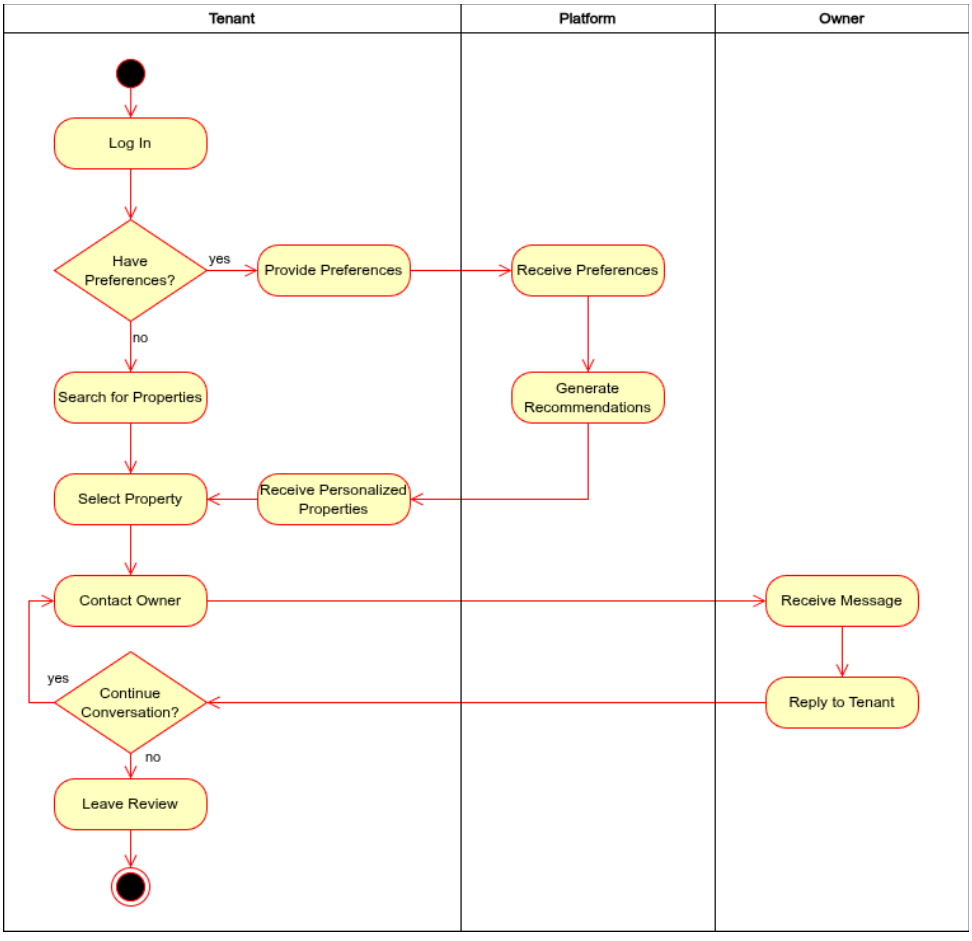

Child Diagram for Process 3 (Property Handler):



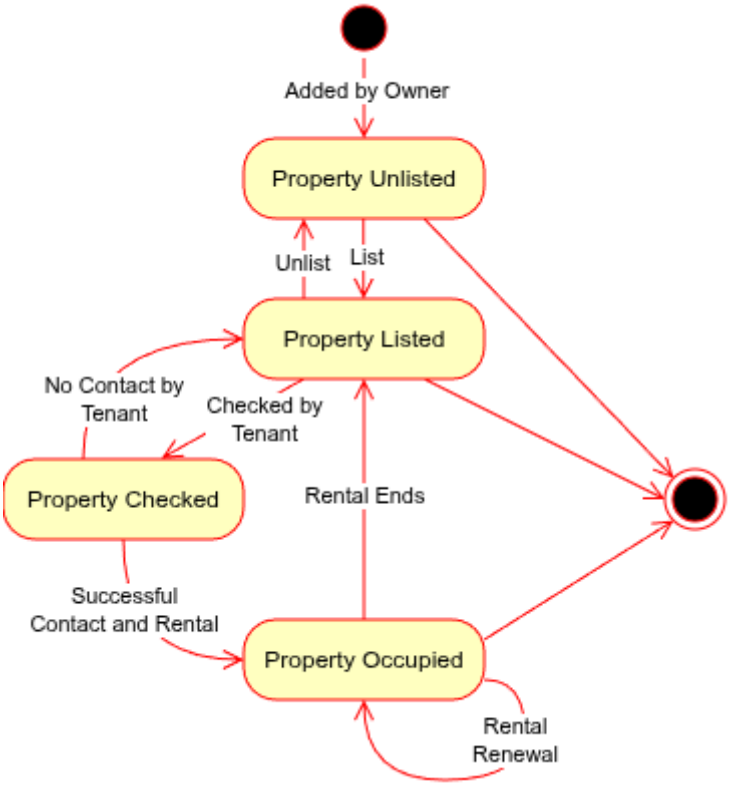
## Sequence Diagram (for Tenant)



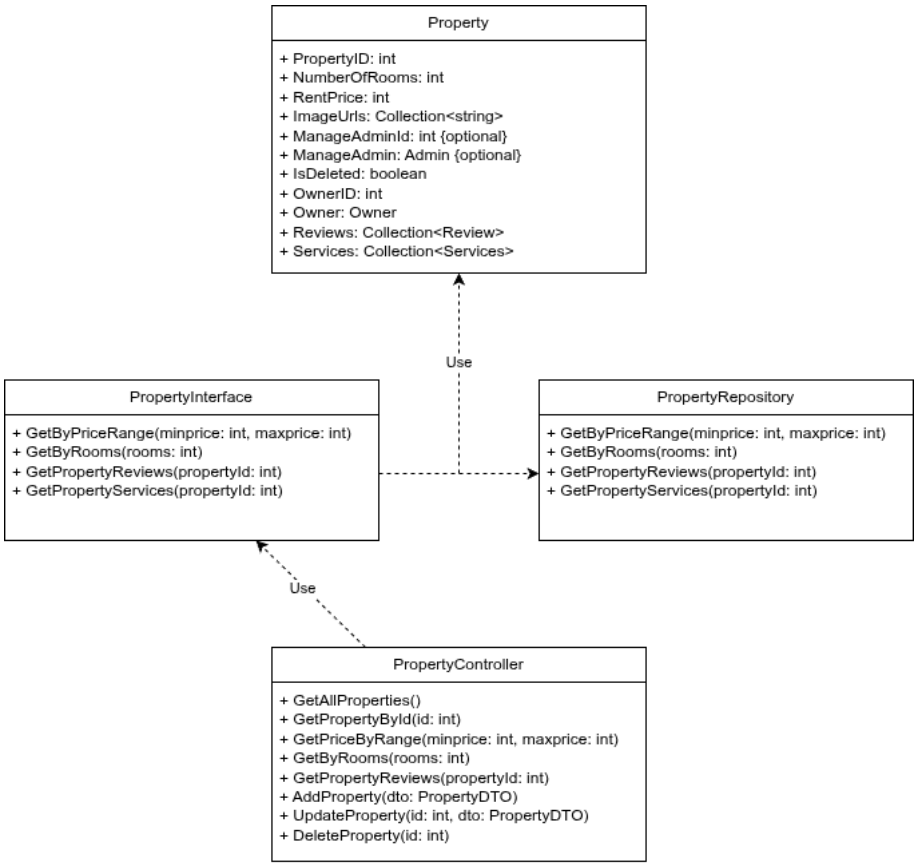

## Activity Diagram (complete with swimlanes, starting from Tenant)



## State Diagram (for Property)

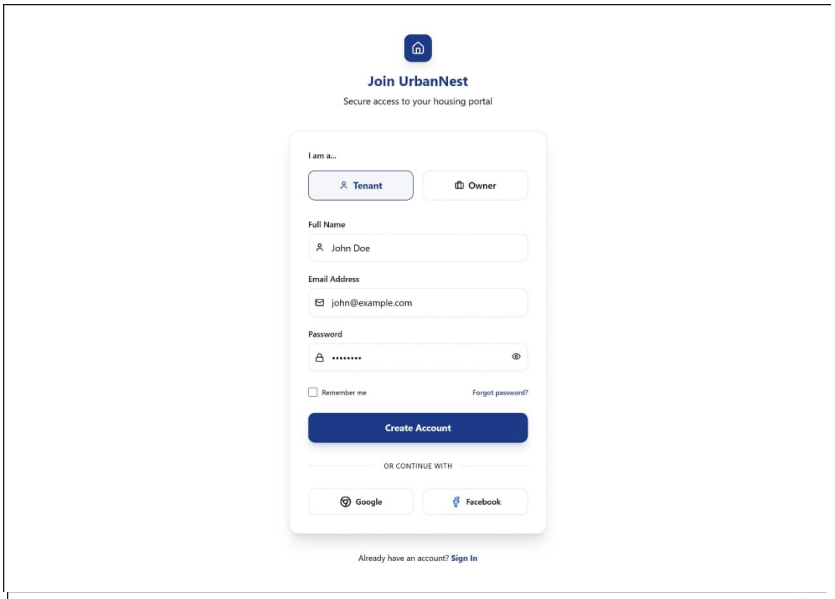


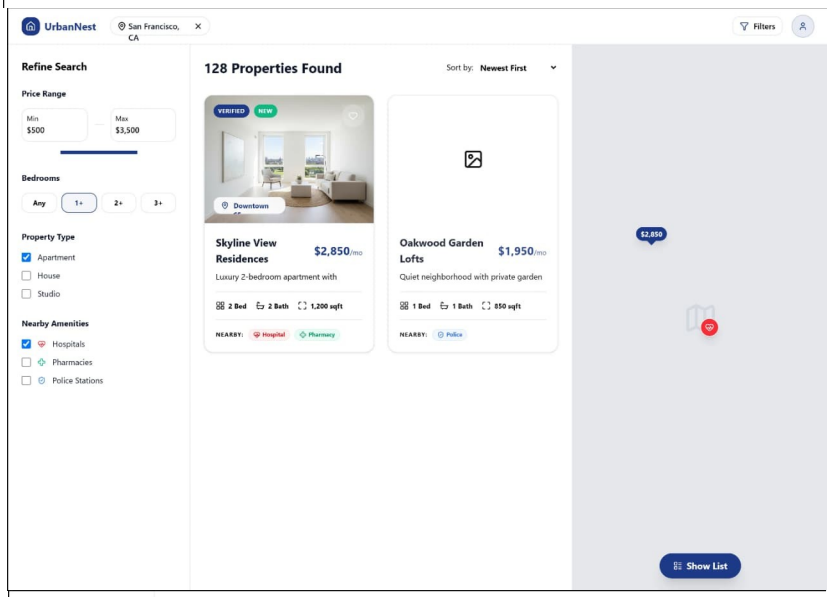

## Class Diagram (for Property, based on three-tier architecture; can be applied retroactively to all entities)

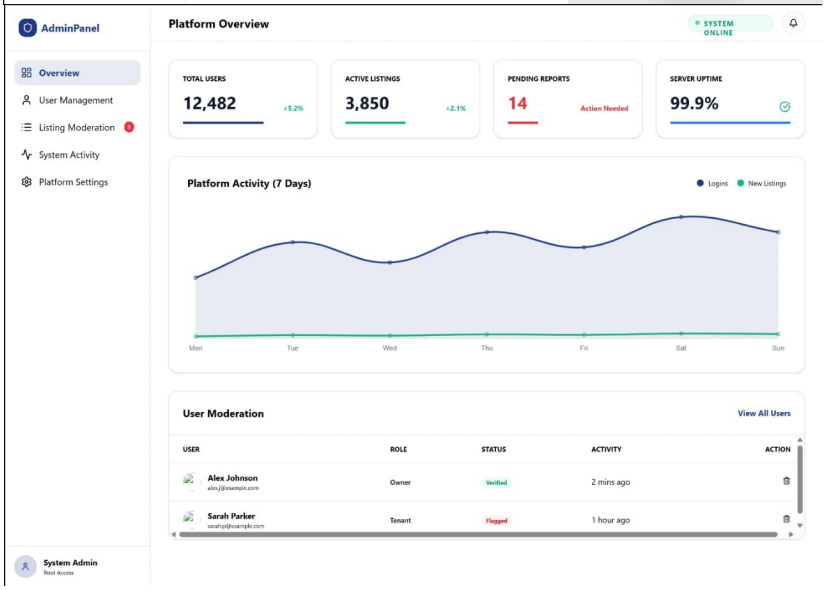


## UI/UX Design & Prototyping

The following mockups were used as general guidelines for the project's frontend design.



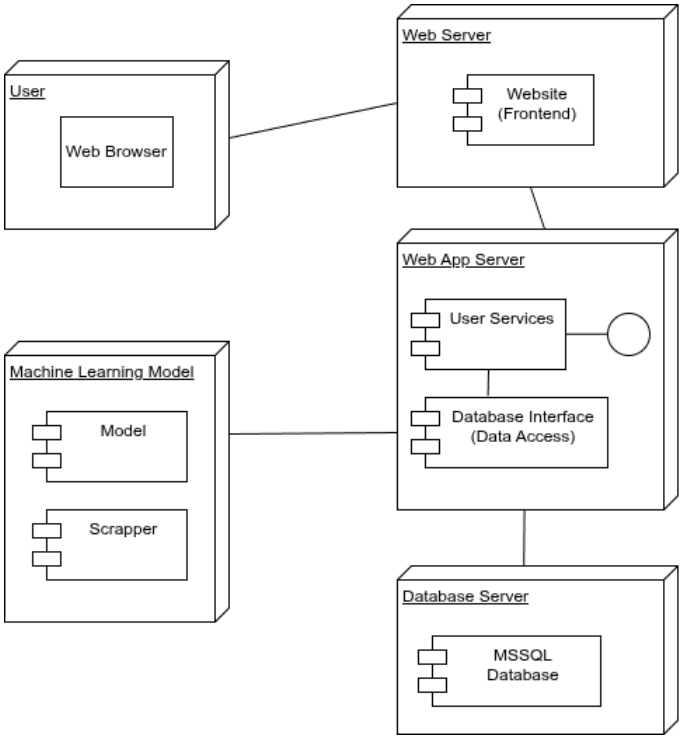
## System Deployment and Integration

### Technology Stack

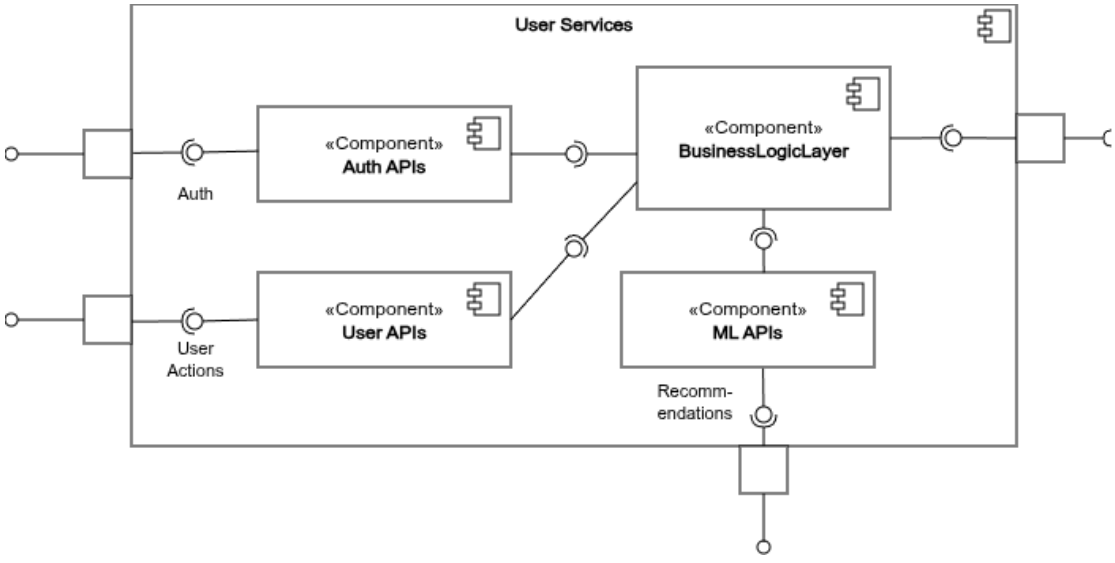
Front-end: React.js
Back-end: .NET 10
Database: Microsoft SQL Server
Machine Learning Service: Python (FastAPI + Celery + Redis)

### Deployment Diagram

During the whole project, each of the 4 entities in the stack would be developed separately, preferably in separate containers.




### Component Diagram (for User Services, integrating both Business Logic and Presentation layers in the web app)



## Additional Deliverables

### Backend API Documentation

Admin Controller (/api/AdminApis/)

| Endpoint | Method | Input | Output |
|---|---|---|---|
| properties | GET | None | Properties |
| GetAllAdmins | GET | None | List<AdminDTO> |
| GetAdmin{id} | GET | int id | AdminDTO |
| AddAdmin | POST | AdminDTO | AdminDTO |
| UpdateAdmin{id} | PUT | int id, AdminDTO | Success |
| DeleteAdmin/{id} | DELETE | int id | Success |

AdminMl Controller (/api/admin/ml/)

| Endpoint | Method | Input | Output |
|---|---|---|---|
| scrape | POST | StartScrapeDTO | Success |
| scrape/status/{jobId} | GET | jobId | Success |
| train-model | POST | TrainModelDTO | Success |

Auth Controller (/api/Auth/)

| Endpoint | Method | Input | Output |
|---|---|---|---|
| register/admin | POST | AdminDTO | Success |
| register/owner | POST | OwnerDTO | Success |
| register/tenant | POST | TenantDTO | Success |
| login | POST | LoginDTO | Success |

Message Controller (/api/MessageApis/)

| Endpoint | Method | Input | Output |
|---|---|---|---|
| GetMessagesBetween | GET | ownerId, tenantId | List<SearchMessageDTO> |
| GetUnreadMessages | GET | ownerId, tenantId | List<SearchMessageDTO> |
| GetAllMessages | GET | None | List<SearchMessageDTO> |
| GetMessageByID/{id} | GET | int id | SearchMessageDTO |
| AddMessage | POST | MessageDTO | Created MessageDTO |
| UpdateMessage/{id} | PUT | id, MessageDTO | MessageDTO |
| MarkAsRead/{messageId} | PUT | messageId | Success |
| SoftDeleteMessage/{id} | DELETE | id | Success |

Owner Controller (/api/OwnerApis/)

| Endpoint | Method | Input | Output |
|---|---|---|---|
| GetAllOwners | GET | None | List<OwnerDTO> |
| GetOwnerById/{id} | GET | id | OwnerDTO |
| GetOwnerProperties/{ownerId} | GET | ownerId | Properties |
| GetOwnerMessages/{ownerId} | GET | ownerId | Messages |
| AddOwner | POST | OwnerDTO | OwnerDTO |
| UpdateOwner | PUT | id, OwnerDTO | OwnerDTO |
| DeleteOwner/{id} | DELETE | id | Success |

Preferences Controller (/api/Preferences/)

| Endpoint | Method | Input | Output |
|---|---|---|---|
| GetAllPreferences | GET | None | List<PreferencesDTO> |
| GetPreferenceById/{id} | GET | id | PreferencesDTO |
| GetPreferenceByTenant/{tenantId} | GET | tenantId | PreferencesDTO |
| GetPreferencesByType/{type} | GET | type | List |
| GetPreferencesByPriceRange | GET | minPrice,maxPrice | List |
| GetPreferencesBySoloOrShared/{value} | GET | value | List |
| AddPreference | POST | PreferencesDTO | Created |
| UpdatePreference/{id} | PUT | id, PreferencesDTO | Updated |
| DeletePreference/{id} | DELETE | id | Success |

Property Controller (/api/PropertyApis/)

| Endpoint | Method | Input | Output |
|---|---|---|---|
| GetAllProperties | GET | None | List<PropertyDTO> |
| GetPropertyById/{id} | GET | id | PropertyDTO |
| GetByPriceRange | GET | minPrice,maxPrice | List<PropertyDTO> |
| GetByRooms/{rooms} | GET | rooms | List<PropertyDTO> |
| GetPropertyReviews/{propertyId} | GET | propertyId | Reviews |
| GetPropertyServices/{propertyId} | GET | propertyId | Services |
| AddProperty | POST | PropertyDTO | PropertyDTO |
| UpdateProperty/{id} | PUT | id, PropertyDTO | Updated |
| DeleteProperty/{id} | DELETE | id | Success |

Recommendation Controller (/api/Recommendation/)

| Endpoint | Method | Input | Output |
|---|---|---|---|
| / | GET | None | PropertyRecommendationDTO |
| similar/{propertyID} | GET | propertyId | Success |
| interact | POST | RecordInteractionDTO | Success |
| preferences | POST | UpdatePreferencesDTO | Success |

Review Controller (/api/Review/)

| Endpoint | Method | Input | Output |
|---|---|---|---|
| getAllReviews | GET | None | List<ReviewDTO> |
| GetReviewBy{id} | GET | id | ReviewDTO |
| GetpropertyReviews/{propertyId} | GET | propertyId | List<ReviewDTO> |
| GettenantReviews/{tenantId} | GET | tenantId | List<ReviewDTO> |
| AddReview | POST | ReviewDTO | Success |
| UpdateReview/{id} | PUT | id, ReviewDTO | Success |
| /{id} | DELETE | id | Success |

Service Controller (/api/ServiceApis/)

| Endpoint | Method | Input | Output |
|---|---|---|---|
| getAllServices | GET | None | List<ServiceDTO> |
| GetById/{id} | GET | id | ServiceDTO |
| GetServiceByType/{serviceType} | GET | serviceType | List<ServiceDTO> |
| GetServicesByProperty/{propertyId} | GET | propertyId | List<ServiceDTO> |
| AddService | POST | ServiceDTO | Service |
| UpdateService/{id} | PUT | id, ServiceDTO | Service |
| DeleteService/{id} | DELETE | id | Success |

Tenant Controller (/api/TenantApis/)

| Endpoint | Method | Input | Output |
|---|---|---|---|
| GetAllTenants | GET | None | List<TenantDTO> |
| GetTenantById/{id} | GET | id | TenantDTO |
| GetTenantReviews/{tenantId} | GET | tenantId | Reviews |
| GetTenantMessages/{tenantId} | GET | tenantId | Messages |
| AddTenant | POST | TenantDTO | TenantDTO |
| UpdateTenant/{id} | PUT | id,TenantDTO | TenantDTO |
| DeleteTenant/{id} | DELETE | id | Success |

User Controller (/api/UsersAPis)

| Endpoint | Method | Input | Output |
|---|---|---|---|
| GetAllUsers | GET | None | List<UserDTO> |
| GetUserById/{id} | GET | id | UserDTO |
| GetUserByEmail | GET | email | UserDTO |
| GetUserByPhone | GET | phoneNumber | UserDTO |
| AddUser | POST | UserDTO | Message + UserDTO |
| UpdateUser | PUT | UserDTO | Success |
| DeleteUser/{id} | DELETE | id | Success |

## Testing and Validation

The required testing deliverables are as follows:

Main Files (.docx):

1. Master Test Plan
2. Test Strategy
3. Scope & Objectives

Test Scenarios (.xlsx)

- Search Filter Scenarios
- AI Predicition Scenarios
- Favourites Test Cases
- Admin Dashboard Test Cases
- Security Test Cases
- API Test Cases

Test Data (.xlsx)

- Valid Test Data
- Invalid Test Data
- AI Test Data

Bug Reports (.xlsx)

- Open Bugs
- Fixed Bugs
- Bugs Screenshots

Test Execution (.xlsx)

- Smoke Test Report
- Regression Test Report
- UAT Test Report
- Execution Status

Traceability Matrix (.xlsx)

Final Reports

- Test Summary Report (.docx)
- Defect Summary Report (.docx)
- UAT Sign-off (.docx)

## Deployment Strategy

Hosting Environment

**Target:** self-managed Linux VM (bare metal or cloud-provisioned — e.g., a single DigitalOcean/Hetzner/EC2/Azure VM), no managed container platform.

**Baseline VM sizing (single-instance, launch scale):**

- 2–4 vCPU, 8 GB RAM, 40+ GB SSD — the ML service's model training/Celery workers are the most memory-hungry component; the .NET API and SQL Server are comparatively light at this scale.
- Ubuntu 24.04 LTS or similar, with Podman installed natively (`dnf install podman` / `apt install podman`, depending on distro).

**Network layout on the VM:**

```
Internet
   │
   ▼
[ Reverse proxy / TLS termination ]  ← only this is publicly exposed
   │
   ├──► React static build (served directly by the proxy, or a lightweight container)
   ├──► .NET API container  (127.0.0.1:5089, internal only)
   │        │
   │        ├──► SQL Server container (internal network only, never exposed)
   │        └──► ML API container (internal network only, never exposed — no auth of its own)
   └──► Redis container (internal only, used by Celery)
```

**Critical point:** the ML service has no authentication per its own docs. It must never be bound to a public-facing port — only the .NET backend should be able to reach it, via a Podman internal network (`podman network create backend-net`), with the ML container's port never published to the host's public interface at all.

**Reverse proxy:** Recommend Caddy or nginx as the single public entry point — terminates TLS (via Let's Encrypt/Certbot), routes `/api/*` to the .NET container, everything else to the React static build. Keeps cert management in one place and avoids configuring HTTPS inside the .NET container itself.

Deployment Plan

### Container images:

Four Podman images, each with its own `Containerfile` (Podman's naming, functionally identical to `Dockerfile`):

1. `backend.Containerfile` — multi-stage build: SDK image compiles `WebApplication1`, runtime image (`mcr.microsoft.com/dotnet/aspnet`) runs it. `jwtsettings.json`/`corssettings.json`/`mlapisettings.json` copied in at build time, populated from CI secrets (never committed).
2. `frontend.Containerfile` — Node image builds the Vite app (`npm run build`), output copied into a minimal static server (nginx or Caddy) image.
3. `ml-api.Containerfile` — as defined in the ML service's own repo (FastAPI + Uvicorn).
4. SQL Server — official Microsoft image, run directly rather than custom-built; data must live on a persistent Podman volume, not the container's writable layer, or a container restart wipes the database.

### Environment/config handling:

Following the same principle already established in the .NET codebase (gitignored settings files, `.env.example` templates): production secrets (JWT signing keys, SQL `sa` password, CORS origin) are injected as Podman environment variables at container run time, sourced from GitHub Actions encrypted secrets — never baked into the image itself.

### Deployment steps (per release):

1. GitHub Actions builds and tags images (Step 4 below)
2. Images pushed to a container registry (GitHub Container Registry — `ghcr.io` — is the simplest choice given you're already on GitHub Actions, avoids standing up a separate registry)
3. SSH into the VM (via a deploy key, GitHub Actions' `appleboy/ssh-action` or similar)
4. `podman pull` the new image tags
5. `podman-compose down && podman-compose up -d` (or a rolling replace of just the changed container — see Step 5, zero-downtime note)
6. Run EF Core migrations against production DB as a one-off container invocation (`podman run --rm backend-image dotnet ef database update ...`), before swapping the running API container to the new image, so the schema is always ready before new code expects it.

CI/CD Pipeline (GitHub Actions)

Two workflows: CI (every push/PR — build & test, no deployment) and CD (on merge to `main` — build, push, deploy).

Scaling Considerations

**Near-term (current scope):**

- Single VM, all containers co-located.

- The one thing worth doing at this scale: keep SQL Server data on a named Podman volume, and back it up on a schedule.

- Set resource limits on containers (`podman run --memory`, `--cpus`) so the ML service's training jobs can't starve the .NET API/DB of resources on the same box.

**If/when traffic grows:**

- Vertical scaling first (bigger VM) is simpler than horizontal for a single-instance setup — usually sufficient for a good while before horizontal scaling is actually needed.

- Horizontal scaling of the .NET API specifically is straightforward if two things are already true: JWT auth is already stateless (already the case; no server-side session, any instance can validate any token), and refresh tokens, once persisted, are stored in the shared SQL Server rather than in-memory per-instance (worth keeping in mind when that piece gets built).

- The ML service is the more likely bottleneck long before the .NET API is — model training and Celery task processing are CPU/memory-intensive in ways simple CRUD API calls aren't. Plan to scale the ML service and its Celery workers independently from the .NET API, on separate resource budgets, rather than assuming they scale together.

- Database is the hardest part of this stack to horizontally scale and the last thing you should try to; a read-replica setup or moving to a managed SQL offering is a bigger architectural jump than adding another API container, and shouldn't be attempted reactively under load pressure.

- Moving beyond a single VM eventually means introducing a load balancer in front of multiple API containers and a shared session-independent architecture — already largely satisfied by the JWT design, which is a good sign this migration path stays open without a redesign.

# 4. Implementation (Source Code & Execution)

## Source Code

### Structure and Architecture:

The backend follows a **three-tier architecture**, split into three .NET projects:

```
PresentationLayer / WebApplication1   ← Controllers, DTOs, API-facing config (Program.cs)
BusinessLogicLayer                    ← Repositories (business logic), interfaces, external service clients
DataAccessLayer                       ← Entities (Classes/ModelContext), EF Core AppContext, migrations
```

Dependency direction flows inward: `PresentationLayer → BusinessLogicLayer → DataAccessLayer`. DTOs are defined once, in `PresentationLayer`, and consumed by controllers only — repositories in `BusinessLogicLayer` work directly with entities from `DataAccessLayer`, keeping business logic decoupled from HTTP-facing request/response shapes.

The frontend is a separate React (Vite) application, communicating with the backend exclusively over its REST API; it never calls the ML recommendation service directly.

### Coding standards & naming conventions:

- **Layer-appropriate naming:** entities in `DataAccessLayer.Classes`, DTOs in `PresentationLayer.DTOs.*` (suffixed `DTO`, e.g. `RegisterTenantDTO`), repositories in `BusinessLogicLayer.Repositories` implementing interfaces from `BusinessLogicLayer.Interfaces`.
- **Repository pattern:** every entity type (`User`, `Admin`, `Owner`, `Tenant`, `Property`, etc.) has a dedicated `I{Entity}Repository` / `{Entity}Repository` pair, all implementing a shared `IGenericRepository<T>` base (CRUD + soft-delete).
- **Consistent constructor-injection style:** implemented across repositories and controllers — `private readonly` fields set in the constructor, matching the project's established convention throughout.

### Security practices

- **Password hashing:** all user passwords (Admin/Owner/Tenant) are hashed with BCrypt before persistence; no plaintext password is ever stored or logged. Password changes are handled defensively (empty/unchanged values are not re-hashed on profile updates).
- **JWT-based authentication:** access tokens are short-lived (15 min default), signed with a dedicated secret, and carry `sub`, `email`, `role`, and `jti` claims.
- **Role-based authorization:** `[Authorize(Roles = "...")]` enforced per-controller and per-action across all API surfaces (Property, Admin-only ML operations, etc.), with controllers defaulting to `[Authorize]` and explicit `[AllowAnonymous]` opt-outs only where a route is genuinely public.

- **Resource-ownership authorization:** a custom `IAuthorizationHandler` (`MustOwnHandler` / `MustOwnRequirement`) enforces that an Owner can only modify their *own* Property listings; Admins bypass this check by design.
- **Input validation:** DTOs use DataAnnotations (`[Required]`, `[EmailAddress]`, `[MinLength]`) so malformed requests are rejected automatically at the model-binding stage, before any business logic executes.
- **CORS:** explicitly scoped to the frontend's origin only (no wildcard origins), required since the frontend and backend run on different local ports/domains.
- **Secrets management:** JWT signing keys, CORS origin, ML API base URL, and DB connection strings are kept in gitignored config files (`jwtsettings.json`, `corssettings.json`, `appsettings.Development.json`), each with a committed `.example` template for onboarding.

Modularity & reusability

- Generic repository base (`IGenericRepository<T>` / `GenericRepository<T>`) eliminates repeated CRUD boilerplate across entities.
- A single reusable authorization policy (`MustOwnResource`-style) handles ownership checks for any entity implementing `IOwned`, rather than one bespoke check per entity type.
- The ML recommendation service is fully abstracted behind `IMlApiClient` — controllers and business logic never construct HTTP calls directly, and the external service's ID scheme (Guid-based) is isolated behind a single mapping utility (`MlUserIdMapper`) rather than leaking into the rest of the codebase.

Error handling

- Centralized `try/catch` in controllers translates domain exceptions (`InvalidOperationException`, `UnauthorizedAccessException`) into appropriate HTTP status codes (409, 401) rather than leaking raw stack traces.
- `[ApiController]` automatic model validation returns structured `400` responses for malformed requests with no manual boilerplate required.

## Version Control and Collaboration

**Github Repository:** `https://github.com/nhahub/NHA-4-109`

**Branching strategy:** currently trunk-based (`main` only), no formal branching model in place yet.

- **Recommended next step:** adopt feature branching (`feature/*` → PR → `main`) once more than one contributor is regularly pushing, to reduce merge conflicts and enable code review before merge.

**Commit history:** descriptive, incremental commits reflecting the layered build-out (auth -> role authorization and ownership checks -> ML integration -> frontend wiring).

## Deployment and Execution

### System requirements

| Component | Requirement |
|---|---|
| .NET SDK | 8.0+ |
| Node.js | 20.x (for the Vite/React frontend) |
| Database | SQL Server (2019+ or mssql/server container image) |
| ML Service runtime | Python 3.11+, Redis (Celery broker) |
| Containerization | Docker or Podman (rootless), for deployment builds |

### Installation steps (local development)

1. Clone the repository

```
$ git clone https://github.com/nhahub/NHA-4-109.git
$ cd NHA-4-109
```

2. Restore and configure backend

```
$ cd WebApplication1
$ dotnet restore
```

Create the following gitignored config files alongside `appsettings.json` (see each `.example` file in the repo for the expected shape):

- `appsettings.Development.json` — `ConnectionStrings:DefaultConnection`
- `jwtsettings.json` — `Jwt:AccessSecret`, `Jwt:RefreshSecret`, `Jwt:Issuer`, `Jwt:Audience`, token lifetimes
- `corssettings.json` — `Cors:ReactAppOrigin` (defaults to `http://localhost:5173` for local dev)
- `mlapisettings.json` — `MlApi:BaseUrl` (defaults to `http://localhost:8000/api/v1`)

3. Apply database migrations

```
$ dotnet ef database update --project ../DataAccessLayer --
startup-project .
```

This creates the target database automatically if it doesn't already exist, provided the configured login has sufficient privileges (configured via appsettings.Development.json).

4. ml

```
$ cd ml
$ python -m venv .venv && source .venv/bin/activate
$ pip install -r requirements.txt
$ alembic upgrade head
$ uvicorn app.main:app --reload --port 8000
```

5. frontend

```
$ cd frontend
$ npm install
```

Create .env

```
VITE_API_BASE_URL=https://localhost:/api
```

## Execution guide (running locally)

1. backend (after `cd backend` from root)

```
$ dotnet run --project WebApplication1
```

Backend Swagger UI (http): https://localhost:5089/swagger

Backend Swagger UI (https): https://localhost:7184/swagger

2. ML (after `cd ml` from root)

```
$ uvicorn app.main:app --reload --port 8000
```

ML Service docs: http://localhost:8000/docs

3. frontend (after `cd frontend` from root)

```
$ npm run dev
```

Frontend dev server: http://localhost:5173

## API documentation

Full interactive API documentation is auto-generated via Swagger and available at `/swagger` when the backend is running in Development mode, including a built-in Authorize button for testing JWT-protected endpoints directly. Key endpoint groups:

| Group | Base route | Notes |
|---|---|---|
| Auth | `/api/Auth` | `register/{admin,owner,tenant}`, `login` |
| Property | `/api/PropertyApis` | Public browsing; Owner/Admin-gated writes; ownership-enforced updates |
| Recommendation | `/api/Recommendation` | Authenticated; proxies the ML service, maps internal user IDs to the ML service's Guid scheme internally |
| Admin ML Ops | `/api/admin/ml` | Admin-only; triggers scraping and model retraining on the ML service |